INTE 201: PROGRAMMING 3 (Structured Programming)

Name: Gueta, Ethan Ulrick D.
Instructor: Sir Victor James Ordoñez
Year and Section: DIT 2-3
Date Passed: October 25, 2026

Activities – Week 5 – Arrays
Single-Dimensional • Multi-Dimensional • Processing • Searching • Sorting • Array of Objects

BASIC

Favorite Things Array

Store and print a small list of personal favorites using a single-dimensional array.

Requirements:

- Declare and initialize a String array with 5 of your favorite things.
- Print each item numbered (1 through 5) using a loop.
- Print the array's total length using .length.

Program Link: https://www.jdoodle.com/ia/29uw-rr2b-gtq6

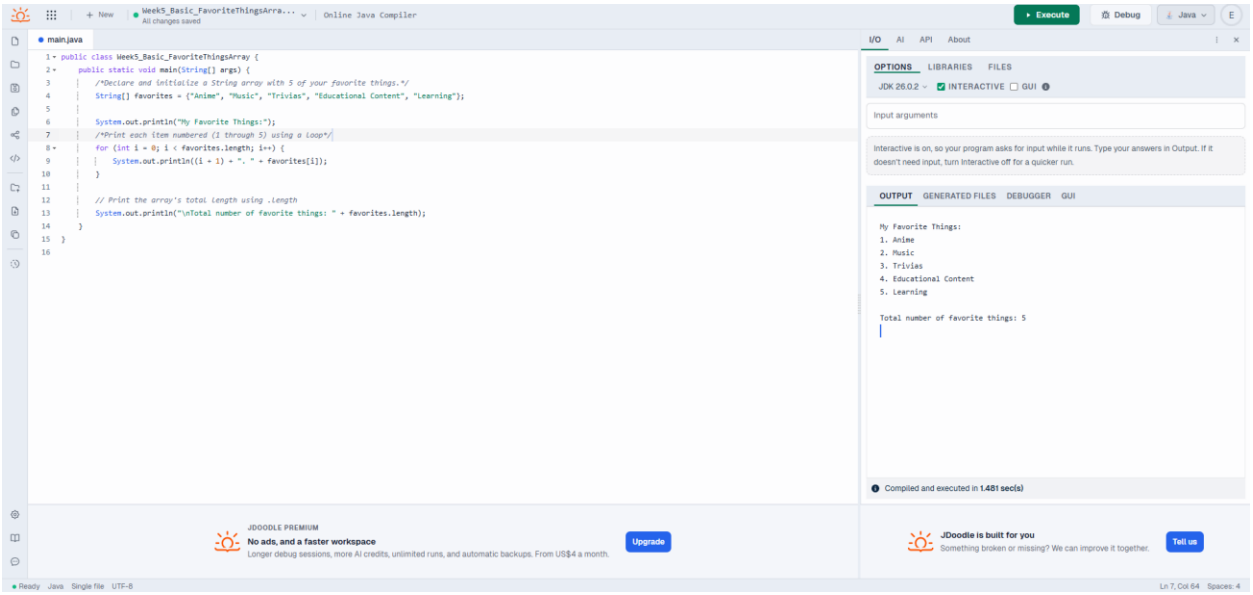
Source Code:

```
public class Week5_Basic_FavoriteThingsArray {
  public static void main(String[] args) {
    /*Declare and initialize a String array with 5 of your favorite things.*/
    String[] favorites = {"Anime", "Music", "Trivias", "Educational Content", "Learning"};

    System.out.println("My Favorite Things:");
    /*Print each item numbered (1 through 5) using a loop*/
    for (int i = 0; i < favorites.length; i++) {
      System.out.println((i + 1) + ". " + favorites[i]);
    }

    // Print the array's total length using .length
    System.out.println("\nTotal number of favorite things: " + favorites.length);
  }
}
```

Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

INTERMEDIATE

Class Statistics Calculator

Process an array of scores to compute several summary statistics.

Requirements:

- Declare an int array of at least 8 student scores.
- Using loops (not built-in shortcuts), compute the sum, average, highest score, and lowest score.
- Print all four statistics with clear labels.

Program Link: https://www.jdoodle.com/ia/e2d5-rb9g-nypu

Source Code:

```
public class Week5_Intermediate_ClassStatisticsCalculator {
  public static void main(String[] args) {
    /*Declare an int array of at least 8 student scores*/
    int[] scores = {99, 92, 75, 88, 95, 70, 89, 94};
```

```
    int sum = 0;
    int highest = scores[0];
    int lowest = scores[0];

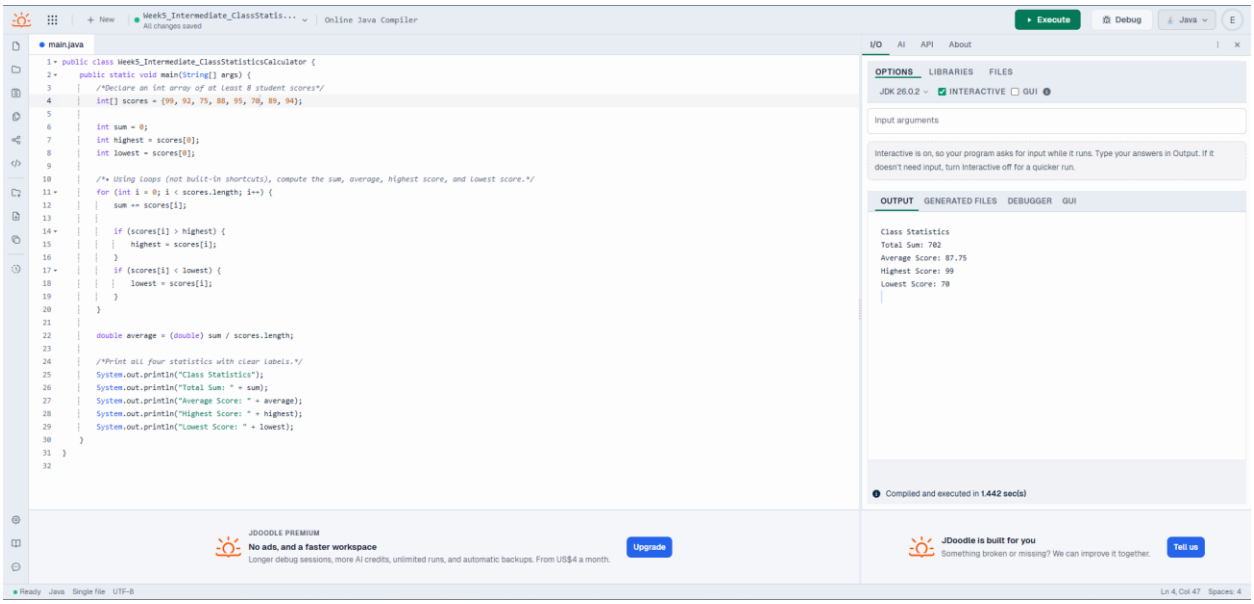
    /*•      Using loops (not built-in shortcuts), compute the sum, average, highest score, and lowest score.*/
    for (int i = 0; i < scores.length; i++) {
      sum += scores[i];

      if (scores[i] > highest) {
        highest = scores[i];
      }
      if (scores[i] < lowest) {
        lowest = scores[i];
      }
    }

    double average = (double) sum / scores.length;

    /*Print all four statistics with clear labels.*/
    System.out.println("Class Statistics");
    System.out.println("Total Sum: " + sum);
    System.out.println("Average Score: " + average);
    System.out.println("Highest Score: " + highest);
    System.out.println("Lowest Score: " + lowest);
  }
}
```

Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

ADVANCED

Search & Sort Mini-Tool

Combine searching and sorting on the same array of data.

Requirements:

- Using the same scores array, implement a linear search for a specific target value, printing whether it was found and at what index.
- Sort a COPY of the array using Arrays.sort(), then print both the original and sorted versions.
- Explain in a comment why sorting a copy (rather than the original) might matter here.

Program Link: https://www.jdoodle.com/ia/bncd-mgk5-ahfs

Source Code:

```
import java.util.Arrays;

public class Week5_Advanced_SearchSortMiniTool {
  public static void main(String[] args) {
    /*Using the same scores array, implement a linear search for a specific
    target value, printing whether it was found and at what index.*/
    int[] scores = {99, 92, 75, 88, 95, 70, 89, 94};
    int target = 88;
    boolean found = false;
    int foundIndex = -1;

    // Linear search for a specific target value
    for (int i = 0; i < scores.length; i++) {
      if (scores[i] == target) {
        found = true;
        foundIndex = i;
        break;
      }
    }

    if (found) {
      System.out.println("Target " + target + " was found at index: " + foundIndex);
    } else {
      System.out.println("Target " + target + " was not found in the array.");
    }

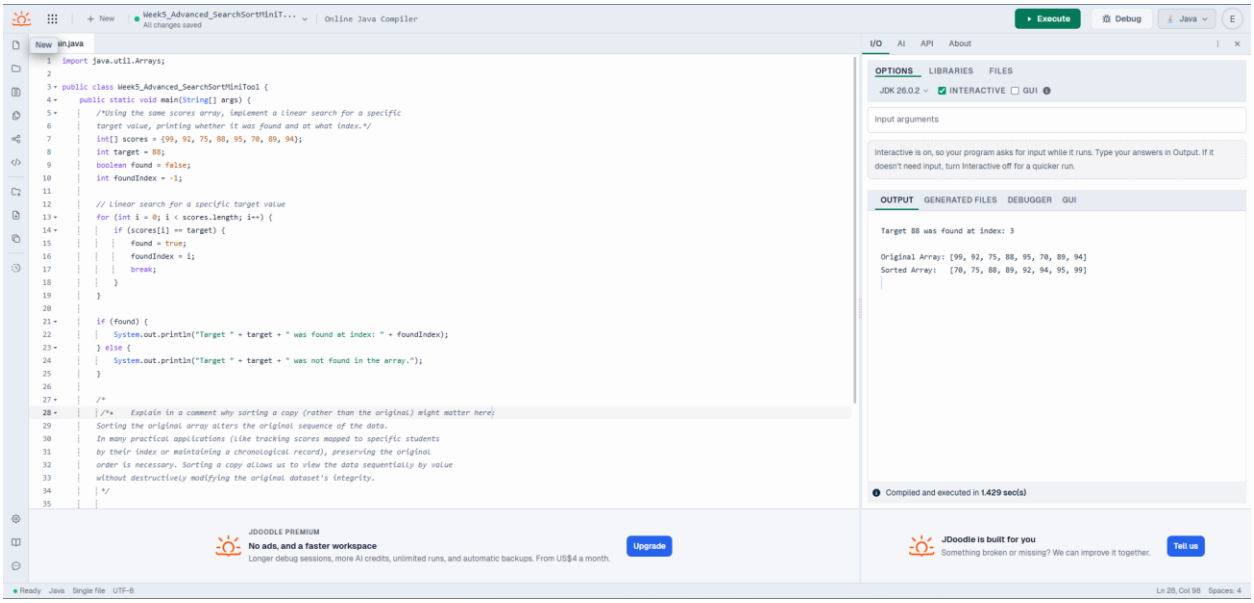
    /*
     /*•  Explain in a comment why sorting a copy (rather than the original) might matter here:
    Sorting the original array alters the original sequence of the data.
    In many practical applications (like tracking scores mapped to specific students
    by their index or maintaining a chronological record), preserving the original
    order is necessary. Sorting a copy allows us to view the data sequentially by value
    without destructively modifying the original dataset's integrity.
     */

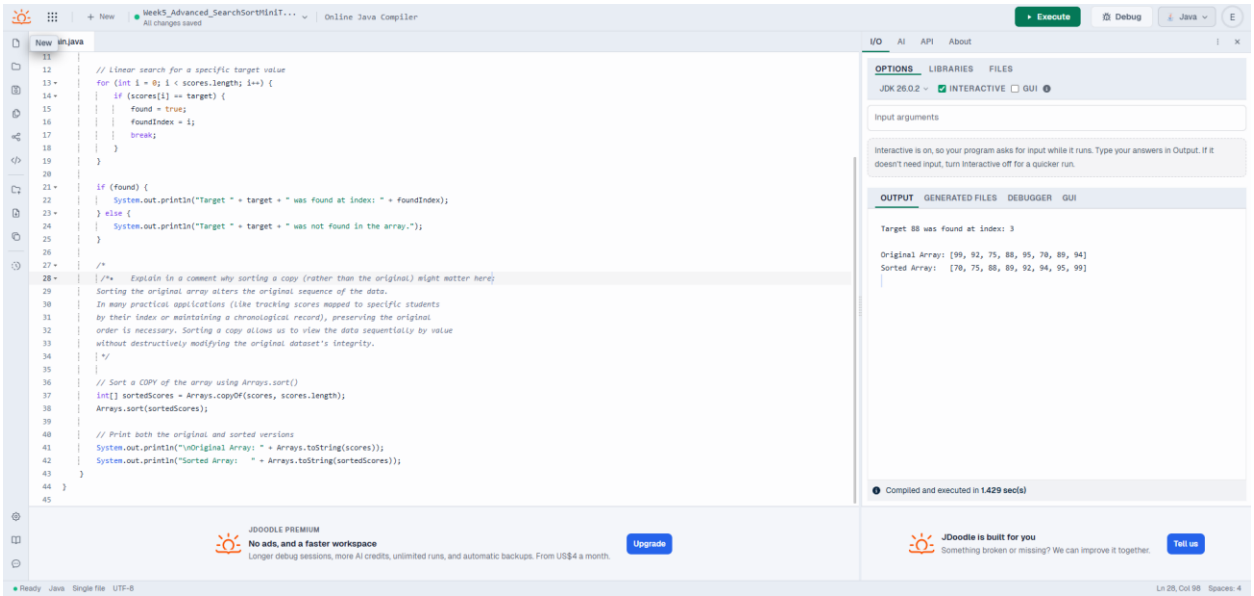
    // Sort a COPY of the array using Arrays.sort()
```

```
    int[] sortedScores = Arrays.copyOf(scores, scores.length);
    Arrays.sort(sortedScores);

    // Print both the original and sorted versions
    System.out.println("\nOriginal Array: " + Arrays.toString(scores));
    System.out.println("Sorted Array:   " + Arrays.toString(sortedScores));
  }
}
```

Screenshots:





Compiler Used:

https://www.jdoodle.com/online-java-compiler

EXPERT

Class Ranking System

Build a complete ranking system using arrays, objects, searching, and sorting together.

Requirements:

- Create a Student class with a name and a quizzes array holding at least 4 scores each.
- Build an array of Student objects from a 2D array of raw quiz data.
- Write your OWN linear search returning a student's rank, and your OWN sort algorithm (no Arrays.sort()) to rank students by average.
- Print a formatted leaderboard and correctly handle a tie between two students with the same average.

Program Link: https://www.jdoodle.com/ia/y2zk-p92g-a5sg

Source Code:

```
class Student {
  String name;
  int[] quizzes;
  double average;

  public Student(String name, int[] quizzes) {
    this.name = name;
    this.quizzes = quizzes;
    this.average = calculateAverage();
  }

  private double calculateAverage() {
    int sum = 0;
    for (int score : quizzes) {
      sum += score;
    }
    return (double) sum / quizzes.length;
  }
}

public class Week5_Expert_ClassRankingSystem {
  public static void main(String[] args) {
    /*Create a Student class with a name and a quizzes array holding at least 4 scores each.*/
    /*Build an array of Student objects from a 2D array of raw quiz data.*/
    String[] names = {"Alice", "Bob", "Charlie", "Diana", "Eve"};
    int[][] rawQuizData = {
      {85, 90, 88, 92}, // Alice
      {78, 82, 80, 85}, // Bob
      {95, 92, 98, 90}, // Charlie
      {85, 90, 88, 92}, // Diana (Tie with Alice)
      {70, 75, 72, 68}  // Eve
```

```
    };

    // Build an array of Student objects from raw data
    Student[] students = new Student[names.length];
    for (int i = 0; i < names.length; i++) {
      students[i] = new Student(names[i], rawQuizData[i]);
    }

    /*Custom sort algorithm (Bubble Sort) to rank students by average (descending)*/
    for (int i = 0; i < students.length - 1; i++) {
      for (int j = 0; j < students.length - 1 - i; j++) {
        if (students[j].average < students[j + 1].average) {
          // Swap
          Student temp = students[j];
          students[j] = students[j + 1];
          students[j + 1] = temp;
        }
      }
    }

    /*Print a formatted leaderboard and correctly handle a tie between two students with the same average.*/
    System.out.println("--- Class Leaderboard ---");
    System.out.printf("%-5s %-10s %-10s%n", "Rank", "Name", "Average");

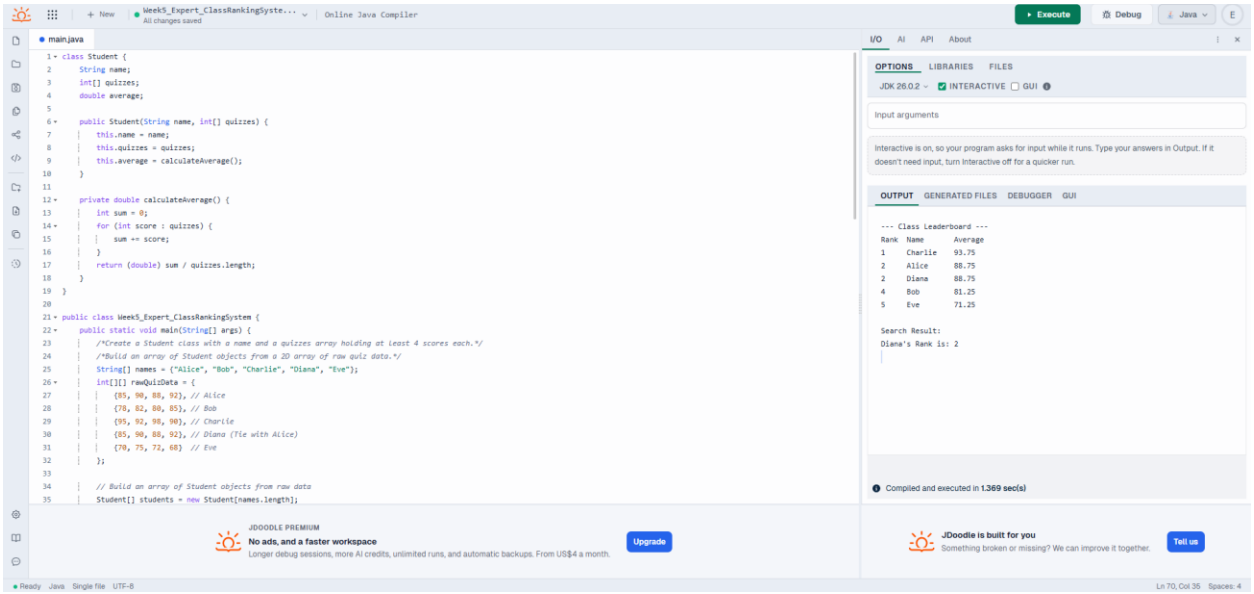
    int currentRank = 1;
    for (int i = 0; i < students.length; i++) {
      // Handle tie: if not the first student and average matches the previous, keep the same rank
      if (i > 0 && students[i].average == students[i - 1].average) {
        System.out.printf("%-5d %-10s %.2f%n", currentRank, students[i].name, students[i].average);
      } else {
        currentRank = i + 1; // Update rank strictly to their position in the sorted array
        System.out.printf("%-5d %-10s %.2f%n", currentRank, students[i].name, students[i].average);
      }
    }

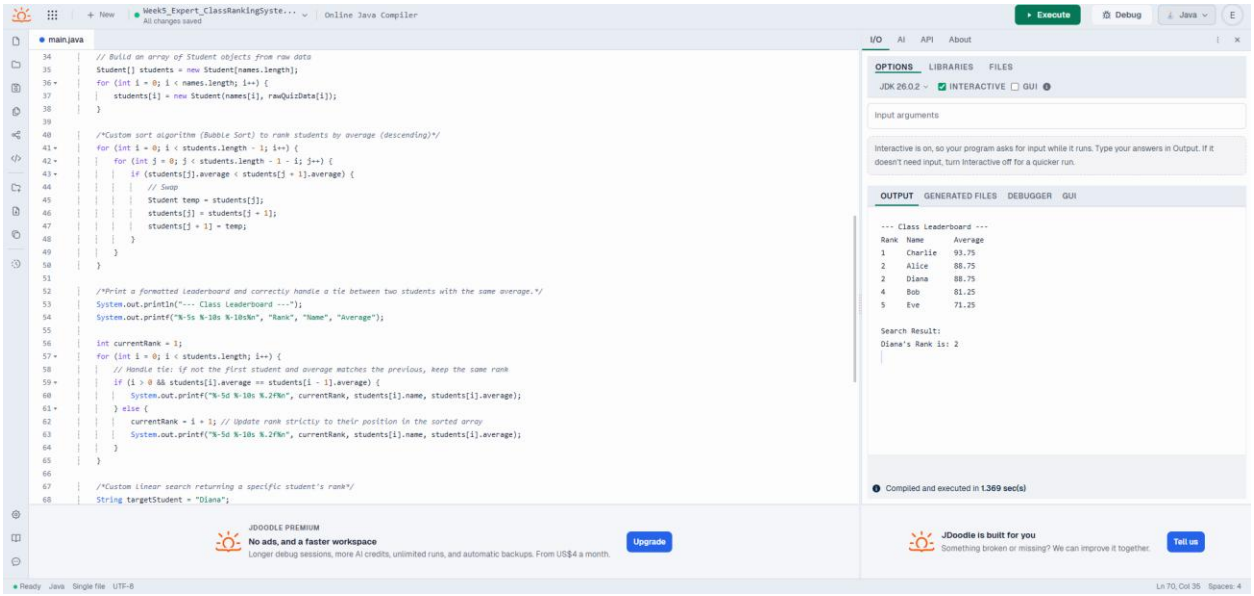
    /*Custom linear search returning a specific student's rank*/
    String targetStudent = "Diana";
    int rank = findStudentRank(students, targetStudent);
    System.out.println("\nSearch Result:");
    if (rank != -1) {
      System.out.println(targetStudent + "'s Rank is: " + rank);
    } else {
      System.out.println("Student not found.");
    }
  }

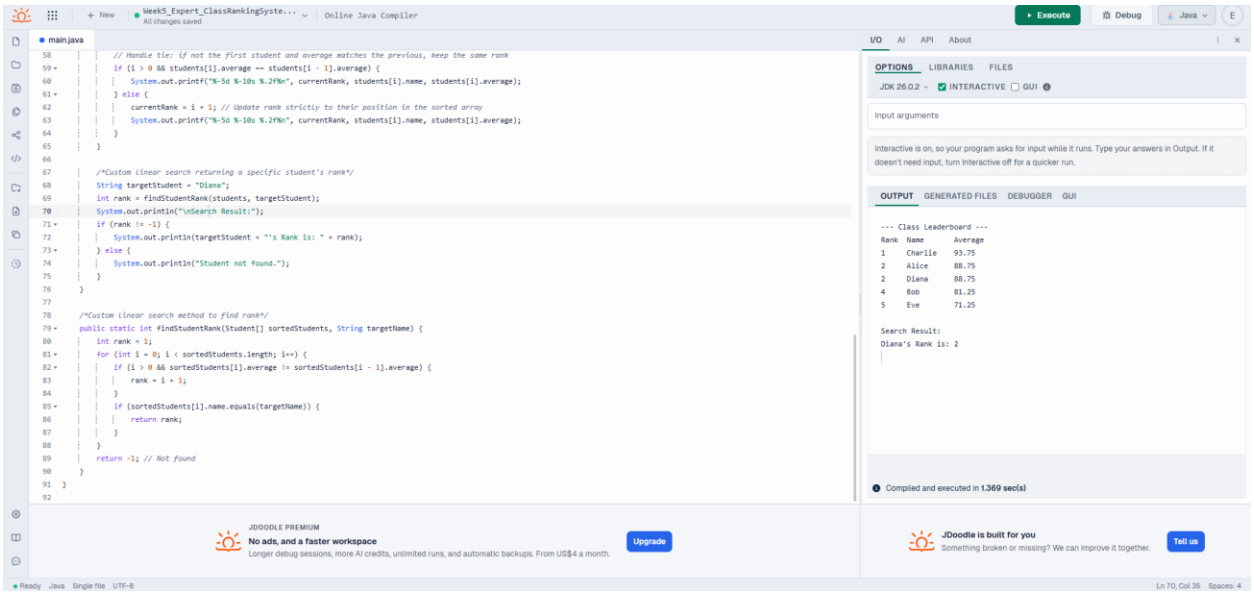
  /*Custom linear search method to find rank*/
  public static int findStudentRank(Student[] sortedStudents, String targetName) {
    int rank = 1;
    for (int i = 0; i < sortedStudents.length; i++) {
      if (i > 0 && sortedStudents[i].average != sortedStudents[i - 1].average) {
        rank = i + 1;
      }
      if (sortedStudents[i].name.equals(targetName)) {
        return rank;
      }
    }
```

```
        return -1; // Not found
    }
}
```

Screenshots:







Compiler Used:

https://www.jdoodle.com/online-java-compiler